Designing a Privacy-Preserving Offline Machine Learning System

For Malicious URL Detection Using URL Features

By Connie Rodriguez

Saint Martin's University

05.07.2026

Agenda

1. Problem Statement & Motivation
2. Research Questions & Objectives
3. System Architecture & Pipeline
4. Experimental Methodology
5. Results & Findings
6. Feature Stability & Model Selection
7. Live Prototype Demonstration
8. Conclusion & Future Work

Saint Martin’s University

01 Problem Statement & Motivation

Why offline, privacy-preserving URL detection matters

Phishing & Malware Threat Landscape

3.4B+	#1	Millions	$10.3B
Phishing emails per day [1]	Leading initial Attack vector [2]	New malicious URLs created daily [1]	Cybercrime losses 2023 [1]

The Deployment Gap

- Most existing systems rely on DNS queries, WHOIS lookups, webpage content, or cloud APIs
- These dependencies are incompatible with air-gapped, privacy-sensitive, and resource constrained environments
- Security tools on endpoint devices, organizational intranets, or classified networks cannot use external services
- The design space of fully offline, URL-string-only detection remains underexplored in the literature

Saint Martin's University

02 Research Questions & Objective

What project aims to achieve, and questions it will answer

Five Core Research Questions

RQ1

Can 24 URL-string-only features, derived exclusively from the URL string, without reliance on DNS queries, WHOIS lookups, or external API calls, support strong phishing detection performance across two independently sourced real-world threat intelligence feeds in a fully offline, privacy-preserving environment?

RQ2

Which URL-string features demonstrate the highest predictive stability across the datasets from different threat intelligence sources?

RQ3

Which lightweight machine learning algorithm provides the best trade-off between classification performance and deployment feasibility for real-world phishing detection?

RQ4

How well does a model trained on PhishTank URLs generalize when evaluated against URLhaus data, and vice versa? What performance gap, if any, exists between the two directions, and what might it reveal about the structural differences between phishing and malware delivery URLs?

RQ5

What measurable improvements over the Iteration 1 baseline can be demonstrated through iterative feature selection and optimization across subsequent experimental rounds?

Project Objectives

OB1

Evaluate the effectiveness of 24 URL-sting-only features for malicious detection across two real-world threat intelligence datasets, operating without any external lookups or network dependencies.

OB2

Assess cross-dataset generalization by training models on one threat source and evaluating on the other in both directions, quantifying the performance gap and identifying structural differences between threat categories.

OB3

Compare Logistic Regression, Random Forest, and Decision Tree across accuracy, precision, recall, and F1-score to determine the best trade-offs between classification performance and deployment feasibility.

OB4

Identify which URL-string features demonstrate the highest predictive stability across both datasets through Random Forest feature importance analysis, informing iterative feature selection.

OB5

Demonstrate measurable improvements over the Iteration 1 baseline through iterative feature selection and hyperparameter optimization, documenting the impact of each round on performance metrics.

Saint Martin's University

03 System Architecture & Pipeline

Eight-script modular, offline, privacy-preserving pipeline

Four Core Design Principles

Reproducibility

Fixed random_state=42 throughout. Re-executing on identical inputs produces identical outputs. Confirmed by running Iterations 2 and 3 twice each.

Single-Responsibility

Each of the 8 scripts performs exactly one transformation. All inter-stage data flows through CSV files, no direct function calls between scripts.

Fail-Fast Validation

All inputs validated at script start. Missing files, schema mismatches, encoding errors, and invalid labels caught before any computation begins.

Offline Enforcement

Zero network connections, DNS queries, WHOIS lookups, or API calls at any stage, from raw data ingestion through inference.

Eight-Script Pipeline Architecture

convert_phishtank.py

Parse PhishTank XML → standardized CSV

convert_urlhaus.py

Parse URLhaus feed → standardized CSV

build_dataset.py

Merge + balance datasets (50/50, 20k rows each)

prepare_data.py

Clean, deduplicate, normalize labels

extract_features.py

Engineer all 24 URL-string features

train_models.py

Train 3 classifiers, 80/20 split, 5-fold cross-validation

cross_dataset_eval.py

Bidirectional cross-dataset generalization

predict.py

Offline inference, majority-vote ensemble

24 URL-String-Only Features

Group A: Length

- url_length
- hostname_length
- path_length
- query_length

Group B: Character Counts

- num_dots
- num_hyphens
- num_underscores
- num_slashes
- num_qmarks
- num_equals
- num_ampersands
- num_at
- num_percent
- num_digits

Group C: Structural / Boolean

- has_https
- has_ip
- has_port
- has_www
- has_at_in_url
- subdomain_depth

Group D: Entropy & Ratios

- url_entropy
- hostname_entropy
- digit_ratio
- special_char_ratio

Saint Martin's University

04 Experimental Methodology

Three progressive iterations, one variable change per iteration

Datasets & Experimental Setup

PhishTank

[4]

10,000 Malicious URLs

Credential-harvesting phishing URLs. Characterized by entropy-based path obfuscation and deceptive domain construction.

URLhaus

[5]

10,000 Malicious URLs

Malware delivery and botnet infrastructure URLs. Characterized by IP-based hosts, numeric patterns, and short structured paths.

Tranco

[6]

10,000 per Dataset URLs

Research-grade legitimate domain rankings. Selected over Alexa for resistance to manipulation through multi-source averaging.

Three Progressive Experimental Iterations

1 Iteration 1
Baseline

Full 24 Features + Baseline Hyperparameters

- 80/20 stratified train-test split (16K / 4K rows)
- 5-fold stratified cross-validation on training set
- Bidirectional cross-dataset evaluation (20K rows each)
- Establishes reference for all subsequent comparisons

2 Iteration 2
Feature Selection

Reduced 13 Features, sole variable change

- RF importance averaged across both datasets
- Threshold = 0.01 mean importance
- 11 of 24 features dropped (<2% total weight)
- Same hyperparameters as Iteration 1

3 Iteration 3
Hyperparameter Tuning

Grid Search, sole variable changed

- GridSearchCV, 5-fold CV scored by F1
- LR: C in {0.01, 0.1, 1.0, 10.0, 100.0}
- RF: n_estimators x max_depth grid
- DT: max_depth x min_samples_split grid

Each iteration changes exactly ONE variable, enabling clean attribution of performance changes

Saint Martin's University

05 Results & Findings

Within-dataset, cross-dataset, and three-way comparisons

Iteration 1: Within-Dataset Baseline Performance

PhishTank + Tranco (4,000 test rows)

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9972	0.9990	0.9955	0.9972
Random Forest	0.9975	0.9980	0.9970	0.9975
Decision Tree	0.9980	0.9985	0.9975	0.9980

URLhaus + Tranco (4,000 test rows)

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9998	1.0000	0.9995	0.9997
Random Forest	0.9998	1.0000	0.9995	0.9997
Decision Tree	1.0000	1.0000	1.0000	1.0000

Key Findings

- All three classifiers achieved F1-scores above 0.997 on both datasets with zero external dependencies
- URLhaus produced higher performance than PhishTank, consistent with structural regularity of malware delivery URLs
- Decision Tree achieved perfect 1.0000 F1 on URLhaus with zero false positives and zero false negatives
- 5-fold cross-validation confirms no overfitting: PhishTank 0.9964 ±0.0011 (LR), 0.9967 ±0.0009 (RF), 0.9962 ±0.0010 (DT)

Directly answer RQ1: URL-string-only features ARE sufficient for strong offline detection

Iteration 1: Cross-Dataset Generalization Key Findings

Direction 1: PhishTank → URLhaus

Model	Precision	Recall	F1
Logistic Regression	0.9989	0.9993	0.9991
Random Forest	0.9999	1.0000	1.0000
Decision Tree	0.9991	1.0000	0.9996

Direction 2: URLhaus → PhishTank

Model	Precision	Recall	F1
Logistic Regression	1.0000	0.9475	0.9730
Random Forest	1.0000	0.9662	0.9828
Decision Tree	1.0000	0.9810	0.9904

The Asymmetry Explained

- Perfect precision 1.0000 in both directions, ZERO legitimate URLs misclassified as malicious
- Recall gap in URLhaus → PhishTank: 5.25% (LR), 3.38% (RF), 1.90% (DT) below perfect
- Root cause: PhishTank url_entropy importance = 0.1005 vs only 0.0078 for URLhaus (10x difference)
- URLhaus-trained models learn low entropy weight → blind to PhishTank's randomized path obfuscation
- This is a genuine structural difference between threat categories, NOT a modeling failure

Directly answer RQ4: Directional asymmetry confirmed and explained through feature stability analysis

Iteration 2: Within-Dataset Impact on Feature Selection

11 Features Dropped (less than 2% total weight) – Impact on Within-Dataset Performance:

Dataset	Model	Full 24 Feature F1	Reduced 13 Feature F1	Δ F1
PhishTank	Logistic Regression	0.9972	0.9972	0.0000
PhishTank	Random Forest	0.9975	0.9977	+0.0002
PhishTank	Decision Tree (depth=10)	0.9980	0.9980	0.0000
URLhaus	Logistic Regression	0.9997	0.9997	0.0000
URLhaus	Random Forest	0.9997	0.9997	0.0000
URLhaus	Decision Tree (depth=10)	1.0000	1.0000	0.0000

Key Findings

- Removing 11 features had negligible impact on within-dataset performance across all three models and both datasets
- Largest change: Random Forest on PhishTank +0.0002 F1 (statistically negligible)
- URLhaus within-dataset performance was completely unchanged across all three models
- Confirms that the 11 dropped features were genuinely redundant with no meaningful discriminative contribution

Iteration 2: Cross-Dataset Impact on Feature Selection

Feature Reduction Impact on Cross-Dataset Generalization in Both Directions:

Dataset	Model	Full 24 Feature F1	Reduced 13 Feature F1	Δ F1	Δ Recall
PhishTank → URLhaus	Logistic Regression	0.9991	0.9995	+0.0004	+0.0007
PhishTank → URLhaus	Random Forest	1.0000	1.0000	0.0000	0.0000
PhishTank → URLhaus	Decision Tree (depth=10)	0.9996	0.9996	0.0000	0.0000
URLhaus → PhishTank	Logistic Regression	0.9730	0.9806	+0.0076	+0.0144
URLhaus → PhishTank	Random Forest	0.9828	0.9829	+0.0001	+0.0001
URLhaus → PhishTank	Decision Tree (depth=10)	0.9904	0.9904	0.0000	0.0000

Key Findings

- Most meaningful improvement: LR URLhaus → PhishTank recall improved from 0.9475 to 0.9619 (+0.0144)
- Removing noise features reduced interference in LR's linear decision boundary improving cross-distribution transfer
- Tree-based models showed negligible change, naturally resilient to irrelevant features through bootstrap sampling
- PhishTank → URLhaus direction already at performance maximum in Iteration 1, no meaningful change expected

Iteration 3: Within-Dataset Performance After Hyperparameter Tuning

Three-Way Within-Dataset Comparison- All Three Iterations, both Datasets:

Dataset	Model	Iteration 1 F1	Iteration 2 F1	Iteration 3 F1	Δ F1 (1→3)
PhishTank	Logistic Regression	0.9972	0.9972	0.9970	-0.0002
PhishTank	Random Forest	0.9975	0.9977	0.9977	+0.0002
PhishTank	Decision Tree (depth=10)	0.9980	0.9980	0.9977	-0.0003
URLhaus	Logistic Regression	0.9997	0.9997	0.9997	0.0000
URLhaus	Random Forest	0.9997	0.9997	0.9997	0.0000
URLhaus	Decision Tree (depth=10)	1.0000	1.0000	1.0000	0.0000

Key Findings

- URLhaus within-dataset performance is completely stable across all three iterations for all three models
- PhishTank shows minor fluctuations within ±0.0003 F1, not operationally significant
- Hyperparameter tuning did NOT degrade within-dataset performance while pursuing generalization improvements
- Iterative process successfully targeted cross-dataset gap without introducing within-dataset regression

Iteration 3: Cross-Dataset Performance- Three-Way Comparison

Three-Way Cross-Dataset Comparison- Both Evaluation Directions:

Dataset	Model	Iteration 1 F1	Iteration 2 F1	Iteration 3 F1	Δ F1 (1→3)	Δ Recall (1→3)
PhishTank → URLhaus	Logistic Regression	0.9991	0.9995	0.9993	+0.0002	+0.0007
PhishTank → URLhaus	Random Forest	1.0000	1.0000	0.9999	-0.0001	0.0000
PhishTank → URLhaus	Decision Tree (depth=10)	0.9996	0.9996	0.9989	-0.0007	0.0000
URLhaus → PhishTank	Logistic Regression	0.9730	0.9806	0.9835	+0.0105	+0.0200
URLhaus → PhishTank	Random Forest	0.9828	0.9829	0.9828	0.0000	0.0000
URLhaus → PhishTank	Decision Tree (depth=10)	0.9904	0.9904	0.9904	0.0000	0.0000

+0.0144

Recall gain
Iteration 2 (LR)

+0.0056

Recall gain
Iteration 3 (LR)

+0.0200

Cumulative recall
gain (LR)

+200

More threats
per 10,000 URLs

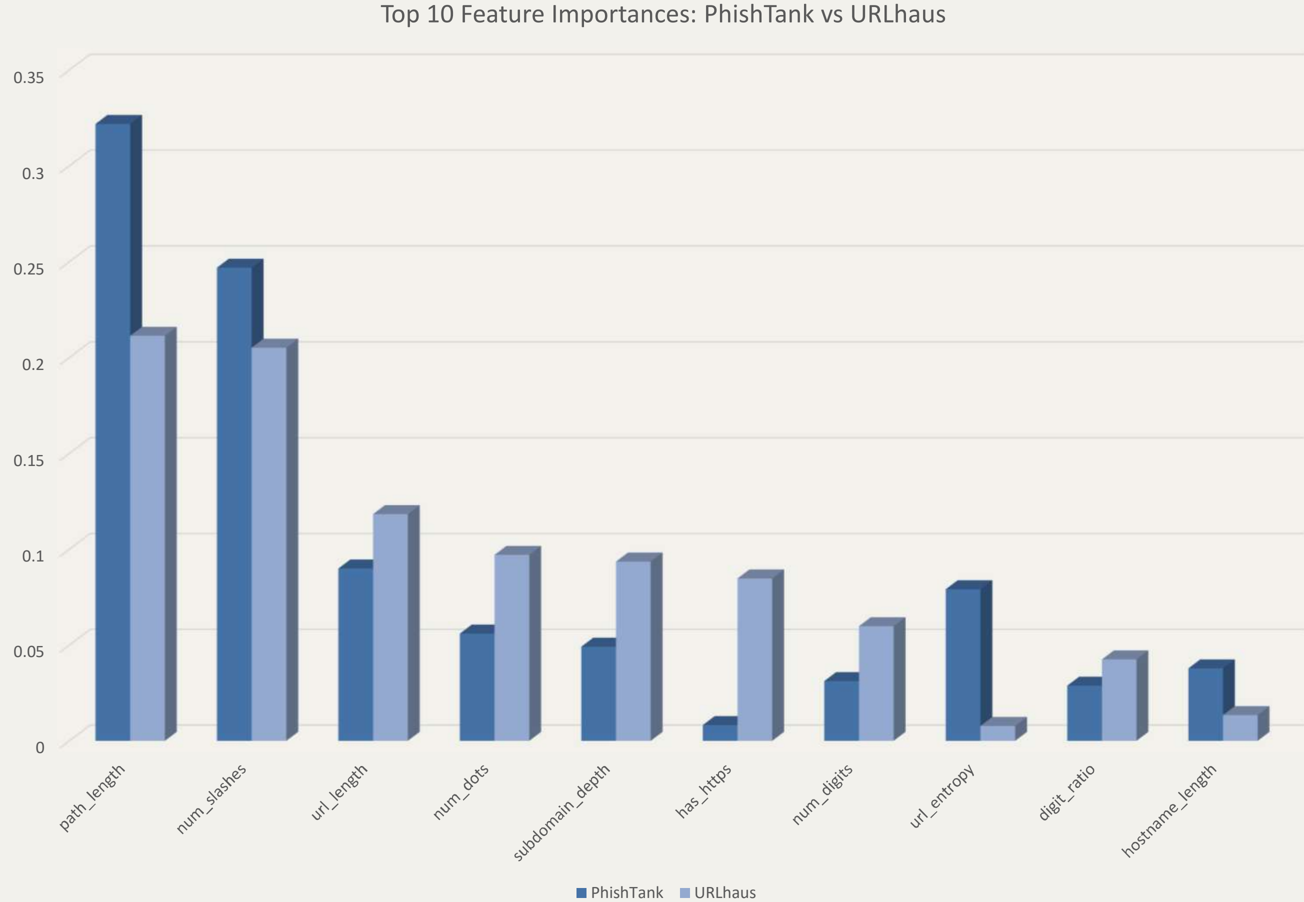
Directly answer RQ5: +0.0200 recall and +0.0105 F1 cumulative improvement through controlled iteration

Saint Martin’s University

06 Feature Stability & Model Selection

What drives generalization, and which model to deploy

Feature Stability Analysis: 24 Features Classified



Key Insights

Stable – Top 2
Path_length + num_slashes = 48.3%
Combined mean weight

url_entropy 10x Gap
0.1005 PhishTank vs 0.0078 URLhaus

has_https Inverse
0.0082 PhishTank vs 0.0847 URLhaus

has_ip URLhaus Only
0.0000 PhishTank vs 0.0216 URLhaus

13 Features Retained
11 dropped – all below 0.01 mean importance

<2% Total Weight
Combined weight of all 11 dropped features

Data Source: feature_importances_comparison.csv

Model Overview:
Roles in the Majority-Vote Ensemble

Decision Tree

Primary Recommendation

- Full auditability, every decision traceable
- Consistent across all 3 iterations
- 0.9904 F1 in hardest generalization direction
- Adapts depth to dataset complexity

```
PhishTank: max_depth=15, min_split=30
URLhaus: max_depth=5, min_split=10
```

Random Forest

Highest Accuracy

- Strongest overall metrics
- 1.0000 F1 PhishTank to URLhaus direction
- Best nonlinear pattern recognition
- Source of feature importance rankings

```
PhishTank: n_est=100, max_depth=10
URLhaus: n_est=100, max_depth=None
```

Logistic Regression

Lightweight Baseline

- Largest improvement across iterations
- +0.0200 recall cumulative gain
- Fastest inference, smallest footprint
- Strongest response to regularization tuning

```
PhishTank: C=100.0
URLhaus cross-eval: C=10.0
```

Deployed inference (predict.py): Majority-vote ensemble of all three classifiers, combining accuracy , interpretability, and efficiency

Saint Martin's University

07 Live Prototype Demonstration

predict.py offline inference with majority-vote ensemble

Saint Martin’s University

08 Conclusion & Future Work

What was achieved, and where the research goes next

Core Research Questions Answered

RQ1

Can 24 URL-string-only features derived exclusively from the URL string, without reliance on DNS queries, WHOIS lookups, or external API calls, support strong phishing detection performance across two independently sourced real-world threat intelligence feeds in a fully offline, privacy-preserving environment?

Yes: F1 > 0.997 on both datasets with zero external dependencies. Offline URL-string-only detection is viable.

RQ2

Which URL-string features demonstrate the highest predictive stability across the datasets from different threat intelligence sources?

path_length and num_slashes most stable (48.3% combined weight). Reduced 13-feature set is sufficient.

RQ3

Which lightweight individual machine learning algorithm provides the best trade-off between classification performance and deployment feasibility for real-world phishing detection?

Decision Tree: best interpretability and performance. Despite this majority-vote ensemble deployed in predict.py for overall balance.

RQ4

How well does a model trained on PhishTank URLs generalize when evaluated against URLhaus data, and vice versa? What performance gap, if any, exists between the two directions, and what might it reveal about the structural differences between phishing and malware delivery URLs?

Meaningful directional asymmetry confirmed. Root cause: url_entropy 0.1005 PhishTank vs 0.0078 URLhaus (10x gap).

RQ5

What measurable improvement in recall and F1 over the Iteration 1 baseline can be demonstrated through iterative feature selection and optimization across subsequent experimental rounds?

Logistic Regression: +0.0200 recall and +0.0105 F1 cumulative over 3 iterations = 200 more threats detected per 10,000 URLs.

Contributions & Future Work

Four Key Contributions

Reproducible empirical baseline for offline, URL-string-only malicious detection.

Quantitative characterization of cross-dataset generalization asymmetry with feature-level explanation.

Demonstration that a 13-feature reduced set matches or outperforms the full 24-feature schema.

Complete modular pipeline with confirmed deterministic behavior and working inference prototype.

Future Work Directions

Explore alternative URL-string representations such as character-level n-grams and token-based segmentation alongside lightweight domain adaptation strategies.

Evaluate combined multi-source training and alternative ensemble strategies such as weighted voting or confidence-based aggregation.

Assess temporal generalization and benchmark the system in practical deployment contexts including constrained hardware and filtering systems.

CONCLUSIONS

This project demonstrates that malicious URL detection can be achieved effectively in fully offline, privacy-preserving environments using 24 URL-string only features and lightweight machine learning classifiers.

- Deliberate, controlled optimization outperforms arbitrary tuning for cross-dataset generalization in constrained detection systems.
- Working prototype confirms system is functionally implementable and accessible for technical and non-technical users in fully offline environments.
- Result is a complete, reproducible system, performing strongly under strict operational constraints with a transparent experimental record that is independently verifiable.

All cited sources and references are documented in the accompanying report.

THANK YOU

Questions

Connie Rodriguez | Saint Martin's University | May 7, 2026 | Advisor: Dr. Dvorak

CSC 598- Master of Science in Computer Science